Machine Learning Basics, Embeddings, and Clustering

Summer School in AI & Applied Economics

Andrea Ciccarone

ETH Zurich

August 31, 2026

Where This Fits

- This is the first substantive lecture of the week: everything later builds on it
- **Today (90 min)**: how predictive models are fit, regularized, tuned, and evaluated out of sample
- **Next block**: how raw text and images become vectors, and how those vectors become research variables
- **Later**: models and representations show up again inside language models and agents

Outline

1. A (Brief) Introduction to Machine Learning
2. Training a Model
3. Flexible Models
4. Finding Structure: Embeddings, Dimension Reduction, and Clustering
5. Wrap-up

Why do we need ML?

- Economists are usually after $\hat{\beta}$: the effect of X on Y *(causal identification)*
- ML is usually after $\hat{Y}$: find prediction function $\hat{f}$ from training data (Y, X) *(prediction)*
- ML is also used as a measurement device for X: it turns high dimensional data into variables you then use in a standard research design *(representation)*
- Two reasons this matters for us
  - Some economic questions *are* prediction problems
  - Some data types (text, images, videos) must first be represented numerically; that is the next block

Supervised vs. Unsupervised Learning

Supervised

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \qquad \widehat{y} = \widehat{f}(x)$$

- We observe a target Y
- Learn a function that predicts Y for new observations
- Examples: house value; whether an ad is Republican

Unsupervised

$$\mathcal{D} = \{x_i\}_{i=1}^N, \qquad Y \text{ is not observed}$$

- Find structure in X itself
- Reduce dimension or group similar observations
- Examples: PCA; k-means clustering

The distinction is simple: **do we observe a target Y?**

ML: Improving from OLS?

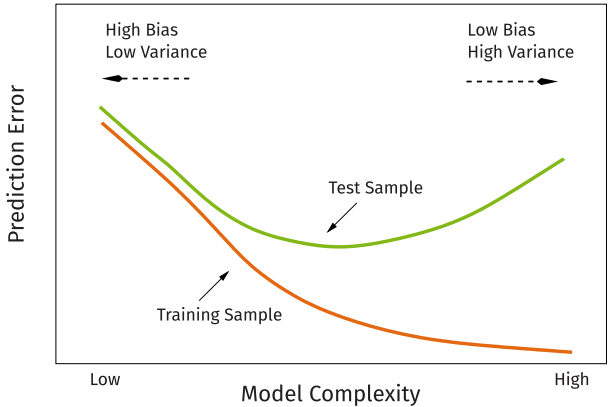
$$Y_i = X_i'\beta + \varepsilon_i, \qquad \mathbb{E}[\varepsilon_i \mid X_i] = 0$$

- OLS is best among *linear unbiased* estimators. Prediction instead asks which estimator minimizes error on a new observation

$$\mathbb{E}\Big[(Y_0 - \widehat{f}(x))^2 \mid X_0 = x\Big] = \underbrace{\sigma^2}_{\text{irreducible noise}} + \underbrace{\Big(\mathbb{E}[\widehat{f}(x)] - f(x)\Big)^2}_{\text{bias}^2} + \underbrace{\text{Var}(\widehat{f}(x))}_{\text{variance}}.$$

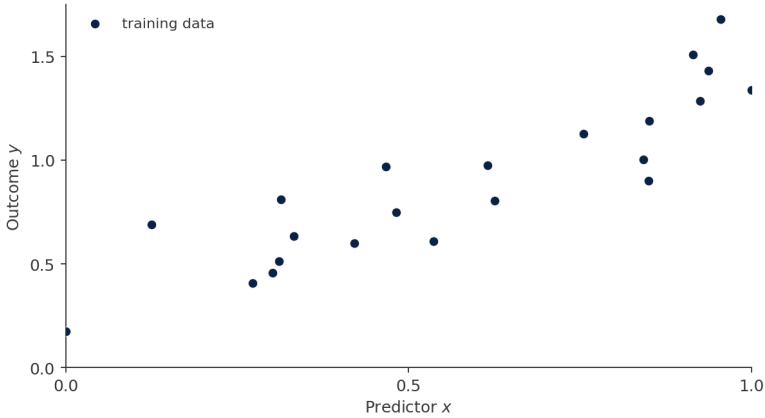
- A very flexible estimator can have low bias but high variance: its prediction changes substantially across training samples
- Regularization deliberately adds bias to reduce variance
- This improves prediction whenever the reduction in variance is larger than the increase in squared bias
- With many covariates, OLS variance can be large; when $p > n$, $X'X$ is not invertible at all

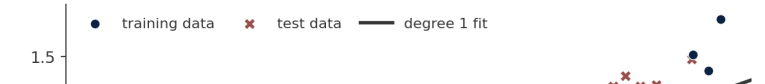
Bias/Variance Trade-off



- More flexibility generally lowers bias and raises variance
- The best complexity minimizes their sum—the expected test error—not either component alone

More Flexibility Can Fit Noise Instead of Signal





Improving from OLS: Ridge

- **Ridge** estimator shrinks coefficients towards 0, reducing sensitivity of single data points
- It introduces bias but lowers variance

$$\begin{aligned}\beta_{Ridge}(\lambda) &= \text{argmin}_{\beta}||y - X\beta||_2^2 + \lambda||\beta||_2^2 \\ &= (X'X + \lambda I)^{-1} X'y\end{aligned}$$

- How do we set λ? **Cross-validation:**
  1. Split train data into K folds
  2. Loop over values of λ
  3. Train model on $K-1$ folds and use the Kth fold for validation
  4. λ^* minimizes average prediction error across folds

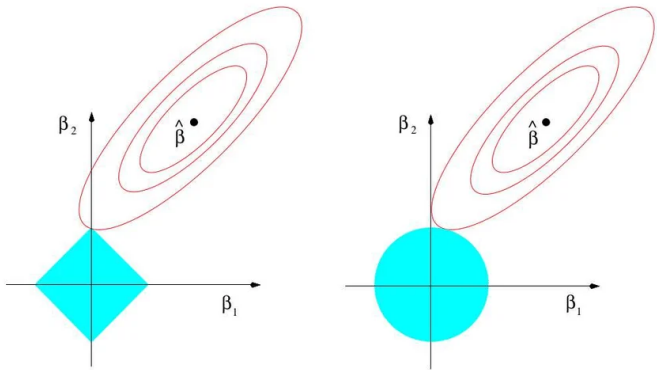
Improving from OLS: Lasso

- The **Lasso** estimator also helps with overfitting. The L_1 penalty sets some coefficients to zero, which can aid **feature selection**

$$\beta_{LASSO}(\lambda) = \text{argmin}_{\beta} ||y - X\beta||_2^2 + \lambda ||\beta||_1$$

- Unlike ridge, the lasso does not have a closed-form solution because the L_1 penalty is not differentiable at zero; use a numerical solver such as coordinate descent or LARS
- λ set, again, with cross-validation

Lasso and Ridge Geometry



- Red ellipses are RSS contours around $\hat{\beta}^{\mathrm{OLS}}$; shaded sets are coefficient-size budgets
- The solution is where the smallest reachable contour first touches the budget set
- The ℓ_1 budget has **axis corners**, so optima often set coefficients to zero; the smooth ℓ_2 ball almost never does

Taking Stock

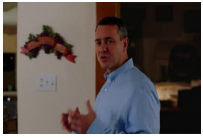
- **High-dimensional data can make traditional estimation unstable or infeasible.** With many parameters, OLS has high variance and overfits; when $p > n$, there is no unique OLS solution
- **Regularization can lower prediction error by reducing variance at the cost of some bias.** Ridge and Lasso are two examples
- **ML models can also give us more flexibility when we need it.** Trees and neural networks can capture nonlinearities and interactions that a linear model misses

Next: how do we train, tune, and evaluate these models out of sample?

Outline

1. A (Brief) Introduction to Machine Learning
2. Training a Model
3. Flexible Models
4. Finding Structure: Embeddings, Dimension Reduction, and Clustering
5. Wrap-up

A Running Example: Is This Ad Republican?


$\ldots$ $\ldots$

$\downarrow$ $\downarrow$ $\downarrow$

$x^1_{img} \in \mathbb{R}^{512}$ $x^t_{img} \in \mathbb{R}^{512}$ $x^N_{img} \in \mathbb{R}^{512}$

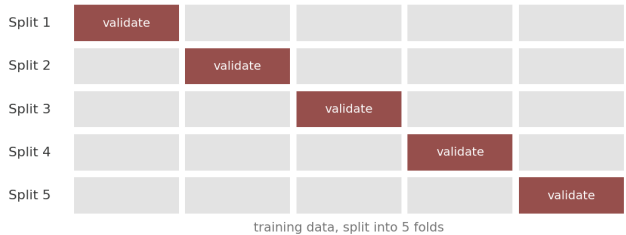
$$\text{One ad} \xrightarrow{\text{some encoder}} \{x^1_{img}, \ldots, x^N_{img}\} \xrightarrow{\text{pooled}} \mathbf{v}_{img} \xrightarrow{\text{classifier}} P(\text{Rep} \mid \mathbf{v}_{img})$$

- Dataset of TV political ads, labelled by the party that paid for them
- Every frame becomes a vector, the ad is the average of its frames, and a classifier turns that into a probability
- *How* an image becomes a vector is the embeddings section. Take it as given for now

Building a Model: Train, Validate, Test

- 3 phases for building our ads model:
  1. **Training**: fit the model weights
  2. **Validation**: choose how many layers, how much dropout - regularization
  3. **Test**: touched once, to report
- **In the ads example, what is an observation?** A frame, or an ad?
- It must be the **ad**. Frames from the same ad share a set, a candidate and a colour grade. Split by frame and the model recognises the ad it already saw, and the reported accuracy is fiction

Cross-Validation



- We do not care whether the model can classify ads it has already seen. We want to know whether it can identify the party of a genuinely new political ad.
- Train on $K - 1$ folds, validate on the held-out one, rotate, then average the K scores
- Every ad is validated exactly once by a model that never saw it. Stack those predictions and you get an **out-of-fold** prediction for the whole sample

What to Report

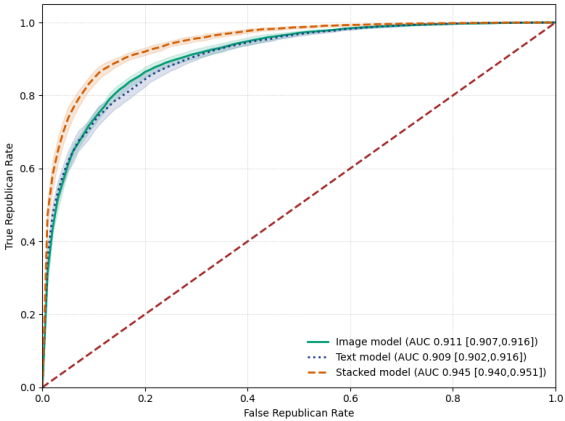
Regression: RMSE, MAE, out-of-sample R^2

Classification:

- **Accuracy** misleads when classes are unbalanced
- **Precision**: of those predicted 1, how many are 1
- **Recall**: of the true 1s, how many we caught
- **AUC**: probability that a random positive outranks a random negative. Threshold-free

Ads are near 50/50, so accuracy is readable there. For imbalanced outcomes, report threshold-free ranking metrics such as AUC alongside precision and recall

The Ads Result



- Three models on the same ads: images only, transcript only, and both
- The image-only model does about as well as the text-only model
- The joint model beats both, so the two signals are **not redundant**
- Every number here is out-of-fold. That is the only reason it means anything

Logistic Regression Is Linear in the Features

For a binary outcome, logistic regression models the **log-odds** linearly:

$$\log\left(\frac{\hat{p}_i}{1-\hat{p}_i}\right) = \beta_0 + x_i'\beta \qquad \Longleftrightarrow \qquad \hat{p}_i = \sigma(\beta_0 + x_i'\beta)\,.$$

- **Linear:** the log-odds and decision boundary are linear in the supplied features
- **Nonlinear probability:** the sigmoid bends the probability, not the feature relationships
- **Interactions are not automatic:** add $x_1 x_2$, polynomials, splines, or learned features
- **Fit:** minimize binary cross-entropy (Bernoulli maximum likelihood); ridge or lasso can regularize

In the ads model: learned features $\longrightarrow$ regularized logistic regression $\longrightarrow$ $P(\text{Republican})$.

Taking Stock: Training a Model

A reliable recipe

1. Choose the loss and evaluation metric
2. Split train, validation, and test at the correct observational unit
3. Fit on training data; tune complexity and regularization with validation or cross-validation
4. Freeze all choices; evaluate once on the untouched test set

Problems to watch for

- **Leakage**: related observations or future information cross splits
- **Validation overfitting**: too many choices are tried on the same validation data
- **Metric mismatch**: accuracy can hide poor performance on rare outcomes
- **Distribution shift**: deployment data differ from the training sample

The test set is for **reporting**, not for making modeling choices.

Outline

1 A (Brief) Introduction to Machine Learning

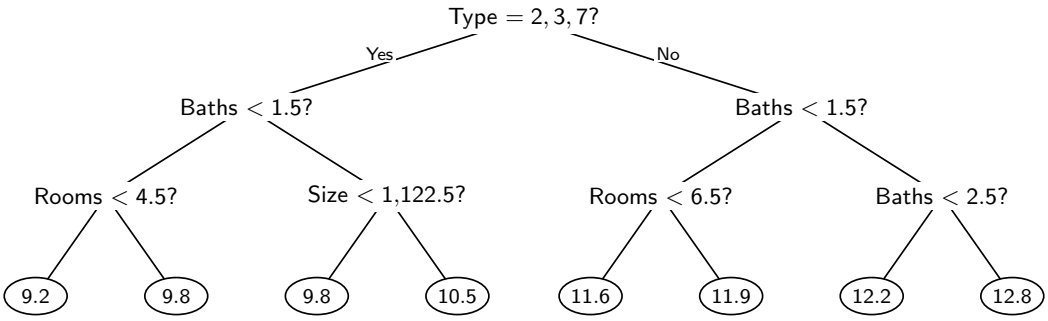
2 Training a Model

3 Flexible Models

4 Finding Structure: Embeddings, Dimension Reduction, and Clustering

5 Wrap-up

A Regression Tree Predicts House Values



Source: Adapted from Mullainathan and Spiess (2017), Figure 1. Predictions are in log dollars.

Follow the questions to a leaf. Its number is the predicted log house value.

How Regression Trees Choose Splits

A tree approximates $f(x)$ by a **step function**.

- Inside a region R, the prediction is just the **sample mean** of the y in it:

$$\hat{y}_R = \frac{1}{|R|} \sum_{i \in R} y_i$$

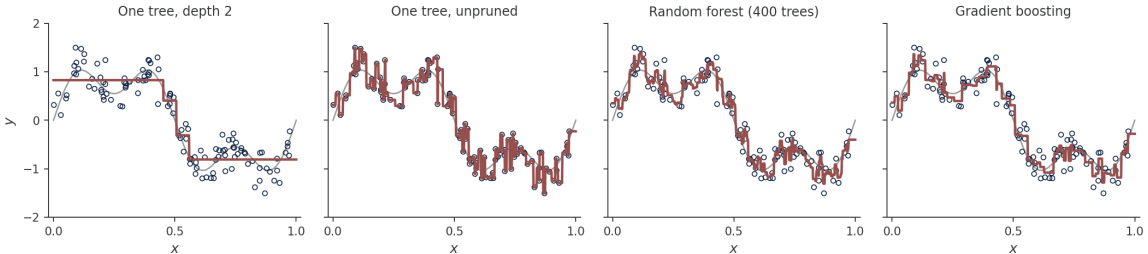
- So the only real question is where to cut. Choose the variable j and threshold c that most reduce the sum of squared residuals:

$$\min_{j,\, c} \sum_{i:\, x_{ij} < c} \left(y_i - \hat{y}_L\right)^2 + \sum_{i:\, x_{ij} \geq c} \left(y_i - \hat{y}_R\right)^2$$

- Then repeat inside each piece. Greedy: each split is chosen as if it were the last

Nothing is assumed about functional form. Interactions and nonlinearities are found, not specified.

From One Tree to Many



- One deep tree chases the noise
- A **random forest** grows many trees on bootstrap samples and averages them. Averaging reduces variance
- **Boosting** grows trees in sequence, each one fitting what the previous ones got wrong

Gradient Boosting in Practice

- Each new tree is fitted to the gradient of the loss left over by the ones before:

$$F_m(x) = F_{m-1}(x) + \nu\, f_m(x)$$

  where ν is the learning rate. Small ν with many trees beats large ν with few

- **XGBoost** and its relatives are the standard implementations, and on tabular data they usually beat a neural network

- **Early stopping** is the regularizer that matters: add trees while a held-out score improves, stop when it does not

If your features are already structured, start here, not with deep learning.

Capturing Complex Relationships: Neural Networks

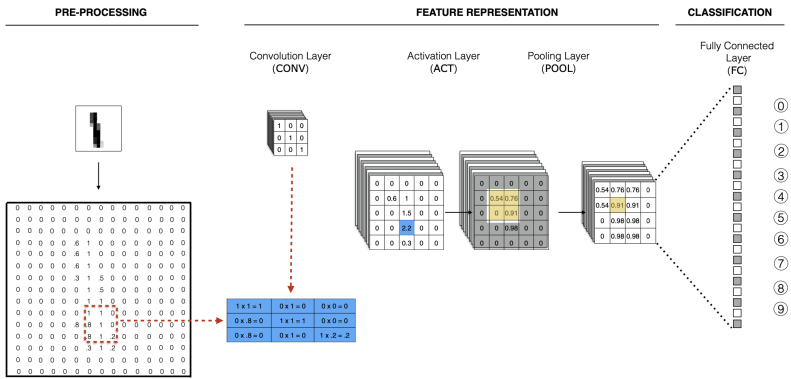
Often, when working with text—and images in particular—we need more complex, nonlinear models.

- A **Neural Network** is a model:

$$g(X;\theta) = f_k(f_{k-1}(\cdots f_2(f_1(X;\theta_1);\theta_2)\cdots;\theta_{k-1});\theta_k)$$

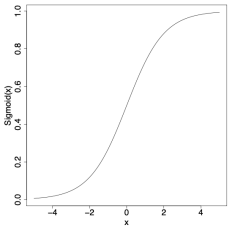
  - Input X is successively transformed through a series of functions (layers) $f_1, f_2, \ldots f_k$
  - Each θ_i is a vector or matrix of weights for layer i (this is the **learning** part)
  - A typical layer may look like $f_i(a_{i-1};\theta_i) = \sigma(W_i a_{i-1} + b_i)$, where σ is often tanh, sigmoid, or ReLU: $\max(0, x)$
  - The output layer can be linear for continuous outcomes, sigmoid for binary classification, or softmax for multiclass classification
  - Our goal is to find the parameters θ that minimize some loss function $\mathcal{L}$

Picture of a (Convolutional) Neural Network

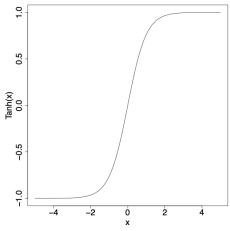


Source: Torres and Cantù (2022)

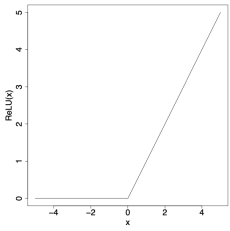
Examples of Activation Functions σ



(a) $Sigmoid(x) = \frac{1}{1+e^{-x}}$



(b) $Tanh(x) = \frac{2}{1+e^{-2x}}$



(c) $ReLU(x) = \begin{cases} 0 & \text{if } x < 0, \\ x & \text{otherwise.} \end{cases}$

Source: Torres and Cantù (2018)

How to Estimate a Neural Network

Forward Pass:

1. Given starting parameters θ, the network applies each layer in sequence: $a_1 = f_1(X; \theta_1)$, $a_2 = f_2(a_1; \theta_2), \ldots, g(X; \theta) = a_k = f_k(a_{k-1}; \theta_k)$
2. Once we have $g(X; \theta)$, we compute the loss $\mathcal{L}\left(g(X; \theta), Y\right)$

Backpropagation:

1. From the last layer, chain rule to compute:

$$\frac{\delta \mathcal{L}}{\delta \theta_k} = \frac{\delta \mathcal{L}}{\delta a_k} \cdot \frac{\delta a_k}{\delta \theta_k}$$

and (recall a_{k-1} is the input of the final layer)

$$\frac{\delta \mathcal{L}}{\delta a_{k-1}} = \frac{\delta \mathcal{L}}{\delta a_k} \cdot \frac{\delta a_k}{\delta a_{k-1}}$$

2. We can then *backpropagate* the gradients through each layer, eg.:

$$\frac{\delta \mathcal{L}}{\delta \theta_{k-1}} = \frac{\delta \mathcal{L}}{\delta a_{k-1}} \cdot \frac{\delta a_{k-1}}{\delta \theta_{k-1}}$$

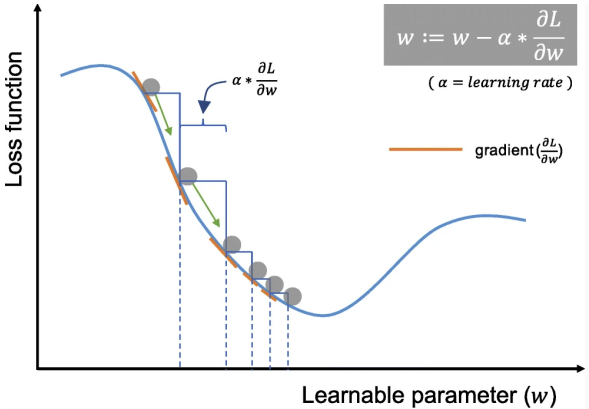
Gradient Descent

- With backpropagation, we can compute each $\frac{\delta \mathcal{L}}{\delta \theta_i}$
- We update parameters θ by moving on the opposite direction of the gradient $\nabla_\theta \mathcal{L}$, because this is the direction of steepest descent
- At each step of the algorithm we update the parameters so that

$$\theta^{t+1} = \theta^t - \alpha \nabla_\theta \mathcal{L}$$

  where α is called the *learning rate*
- We iterate until a convergence criterion is met or validation performance stops improving

Gradient Descent



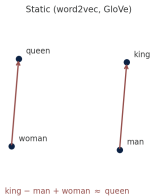
Source: Yamashita et al. (2018)

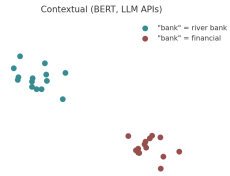
Outline

1. A (Brief) Introduction to Machine Learning
2. Training a Model
3. Flexible Models
4. Finding Structure: Embeddings, Dimension Reduction, and Clustering
5. Wrap-up

Embeddings Turn Raw Objects into Feature Vectors

- An **embedding** maps a complex object—a word, document, or image—to a numerical vector $x_i \in \mathbb{R}^p$
- Distances and directions in that space are designed to preserve useful semantic or visual relationships
- The vectors can be inputs to prediction, dimension reduction, or clustering





Here we treat a pretrained encoder as given. The next lecture shows how text and image embeddings are constructed and validated.

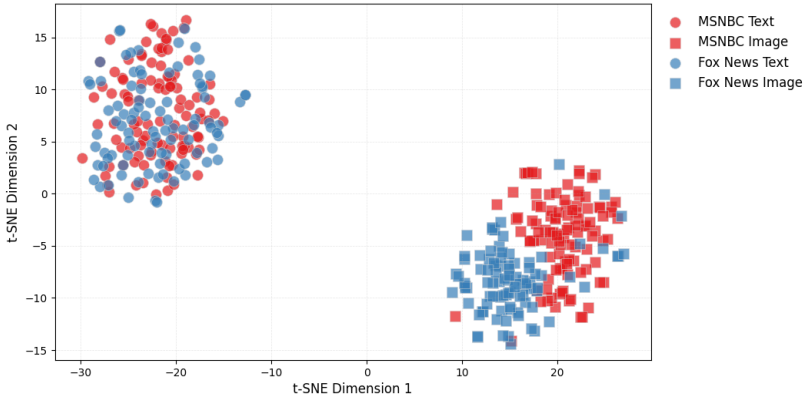
Unsupervised Learning: Finding Structure in X

- In supervised learning we observe a target Y and learn $\hat{f}(X)$
- In unsupervised learning we observe only $X = (x_1, \ldots, x_N)'$, with $x_i \in \mathbb{R}^p$
- We look for a lower-dimensional representation of the variation in X, or groups of similar observations
- **PCA**: replace p variables with a small number of continuous components
- k-**means**: replace N observations with membership in K groups
- For now, take the numerical vectors x_i as given. The next block explains how text and images become vectors

Unsupervised Tasks: PCA

- PCA finds a lower-dimensional representation that retains as much of the variance as possible
- Approximate each observation with $x_i \approx \mu + V_q \lambda_i$, where V_q is a $p \times q$ matrix mapping q component scores into the original p-dimensional space
- Solve $\min_{\lambda, V_q, \mu} \sum_{i=1}^{N} \|x_i - \mu - V_q \lambda_i\|^2$ subject to $V_q' V_q = I_q$
- Compute the singular value decomposition of the centered data matrix: $X_c = UDV^T$
- The columns of V are the eigenvectors of $X_c^T X_c$ and give directions in the original feature space, ordered by explained variance
- The first component captures the most variance; the first q components retain the greatest possible variance among linear q-dimensional projections
- PCA is a linear dimension-reduction method. Nonlinear visualization methods include t-SNE and UMAP

Nonlinear Visualization: t-SNE Example



Unsupervised Tasks: k-means clustering

- Another useful technique is k-means clustering, used to group observations into clusters
- Within each cluster, points are as similar as possible to each other
- We solve:

$$\min_{S} \sum_{j=1}^{K} \sum_{i \in C_j} ||x_i - S_j||^2$$

- We are minimizing within-cluster variance (K is the number of clusters), where S_j is the cluster center
- The centers summarize clusters but are not necessarily observed data points; inspect nearby observations to interpret each cluster

Outline

1. A (Brief) Introduction to Machine Learning
2. Training a Model
3. Flexible Models
4. Finding Structure: Embeddings, Dimension Reduction, and Clustering
5. Wrap-up

Recap

Core logic

1. Use a flexible functional form
2. Limit its expressiveness through regularization
3. Tune how much to regularize using validation data

Important choices

- Loss function
- Data split
- Function class
- Regularizer

Implementation: Training the Ads Classifier

Predict whether an ad is Republican from its CLIP embedding:

```
X_train, X_test, y_train, y_test = train_test_split(
    ad_embeddings, party, test_size=.20,
    stratify=party, random_state=44)

model = keras.Sequential([
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(.20),
    keras.layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=[keras.metrics.AUC(name="auc")])

model.fit(X_train, y_train,
          validation_split=.20, epochs=50,
          callbacks=[keras.callbacks.EarlyStopping(
              monitor="val_auc", mode="max", patience=4,
              restore_best_weights=True)])

test_auc = model.evaluate(X_test, y_test)[1]
```